USER GUIDE

OpenCode Stack

Full User Guide

Mohamed Rafaa Tounekti
DevOps & DevSecOps Engineer

Six tools. One workflow.

OpenCode | oh-my-opencode-slim | OpenSpec | Ponytail | OpenWolf | Herdr

linkedin.com/in/mohamed-rafaa-tounekti · github.com/rafaabou · rafaabou@proton.me

2026-10-30 · OpenCode 2.2.1 · OpenSpec 1.14.0 · Ponytail 5.3.3

Contents

1. The stack at a glance 3

2. Install from zero 3

Where each piece registers 5

3. Where everything lives 5

4. OpenCode — the host 6

Start 6

Inside the TUI 6

Config 7

5. oh-my-opencode-slim — the agent team 7

Agents 7

Configuration 7

Calling agents 8

6. OpenSpec — decide what to build 8

Initialize a project 8

Core loop 8

Skills inside OpenCode 8

Example 9

7. Ponytail — decide how little to build 9

Levels 9

Commands 10

Defaults 10

`shortcut:` comments 10

8. OpenWolf — project memory 10

9. Herdr — terminal workspace manager 11

Start 11

Core objects 11

Agent control (the useful part) 12

Remote and multi-machine 12

10. Security and permissions — what the agent may touch 12

10.1 The trust model 12

10.2 A starter policy 12

10.3 Keys and secrets 13

10.4 MCP servers are context and trust 13

10.5 What this does not do 14

11. When not to use the stack 14

12. Working with all six together — maximum gain 15

12.1 Who decides what 15

12.2 The loop 15

12.3 Weight the process to the task 15

12.4 Scenario A — small fix (minutes) 16

12.5 Scenario B — feature with a spec 16

12.6 Scenario C — parallel lanes (big change) 16

12.7 Scenario D — the bug that keeps coming back 17

12.8 Power combos — what each pairing buys you 17

12.9 Phrasebook — what to type 18

12.10 Guardrails — what not to do 18
12.11 Routine 18
12.12 Which tool when — the one-line rule 19
13. Cheat sheet 19
14. Troubleshooting 21
15. Appendix A — templates to copy 21
A. Minimal `opencode.jsonc` 21
B. Project `AGENTS.md` 22
C. `/security-review` command 22
D. OpenSpec change skeleton 22
16. Appendix B — glossary 23
17. Appendix C — versions, licenses and sources 24

A practical field guide to running six AI tools as one workflow: **OpenCode**, **oh-my-opencode-slim**, **OpenSpec**, **Ponytail**, **OpenWolf** and **Herdr**. Install, commands, file locations, security, templates, and the workflows I use daily. Versions reflect October 2026.

1. The stack at a glance

Tool	Role	Runs as
OpenCode	The AI coding host: chat, agents, tools, models	`opencode` CLI/TUI (v2.2.1)
oh-my-opencode-slim	Agent team + presets on top of OpenCode (orchestrator, explorer, oracle, designer, fixer, observer, council)	TUI plugin + agent presets
OpenSpec	Spec-driven development: proposals, specs, tasks, archive	`openspec` CLI (1.14.0) + OpenCode skills
Ponytail	Coding style: smallest complete change, no over-engineering	OpenCode plugin (level: full)
OpenWolf	Project memory: file anatomy, bug fixes, do-not-repeat rules	OpenCode plugin (`.wolf/`)
Herdr	Terminal workspace manager: sessions, tabs, panes, git worktrees, agent control	`herdr` CLI + server

Mental model: **Herdr** holds the terminals, **OpenCode** is the agent inside each pane, **oh-my-opencode-slim** gives that agent a team, **OpenSpec** decides *what* to build, **Ponytail** decides *how little* to build, **OpenWolf** remembers what was learned. Section 12 is the worked version of this loop.

---

2. Install from zero

Order matters: host first, then plugins, then the companion CLIs. These commands reflect the versions in Appendix C; check `--version` after each step.

Prerequisites: Linux x86_64, `bash`, `curl`, `git`, and Node 20+ for the npm-installed CLIs and plugin hooks. Docker, OpenTofu and Ansible are optional — install them only if your work needs them.

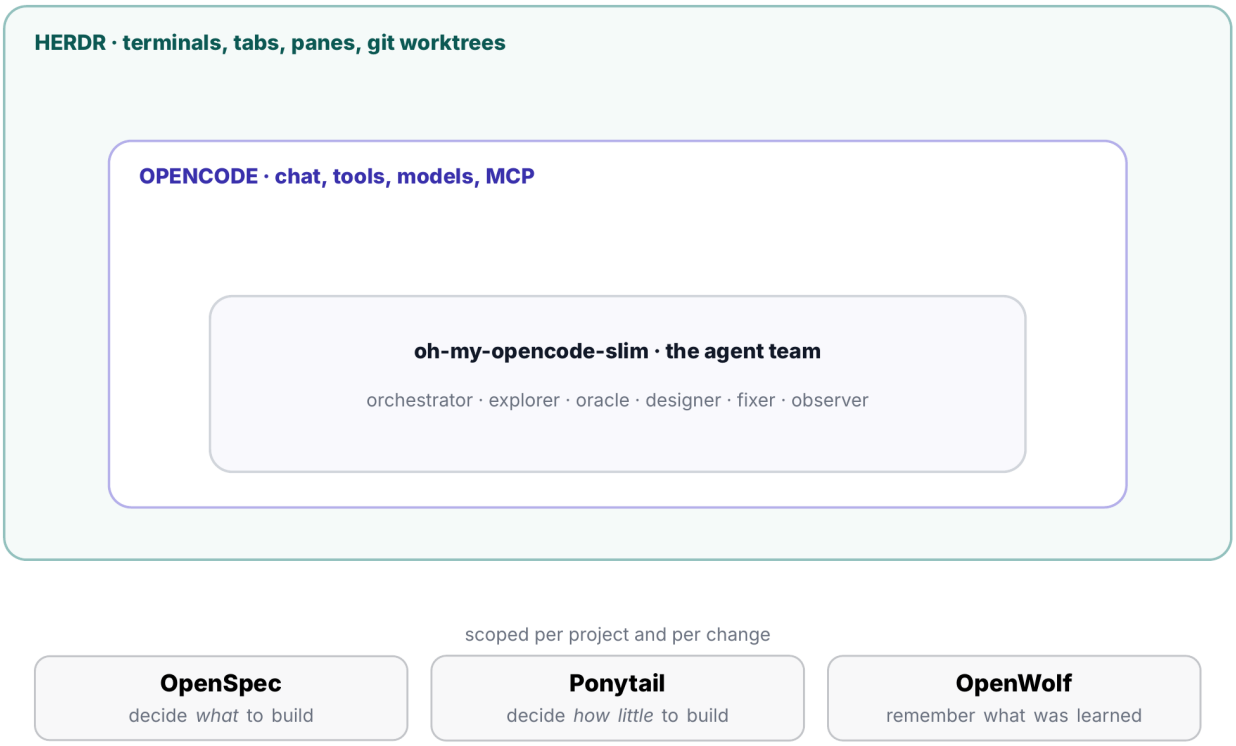


Figure 1: The stack: Herdr holds OpenCode, OpenCode runs the agent team, and the three disciplines scope the work.

```
# 1. OpenCode — the host
curl -fsSL https://opencode.ai/v2/install | bash

# 2. OpenSpec — specs and change artifacts
npm install -g @fission-ai/openspec@latest

# 3. oh-my-opencode-slim — the agent team
bunx oh-my-opencode-slim@latest install
#    no Bun? npx oh-my-opencode-slim@latest install

# 4. Ponytail — coding discipline
opencode plugin add @dietrichgebert/ponytail

# 5. OpenWolf — project memory
npm install -g openwolf
openwolf init --agent opencode

# 6. Herdr — terminal workspace
curl -fsSL https://herdr.dev/install.sh | sh
```

Then verify:

```
opencode --version      # v2.2.1
openspec --version      # 1.14.0
herdr --version         # 0.9.3
herdr status            # client/server state, not a version check
```

Where each piece registers

Tool	Registers as	Lives in
OpenCode	Host binary	`~/.local/bin/opencode`
oh-my-opencode-slim	TUI plugin (installer may also add it to `plugins`)	`tui.jsonc` → `plugin`, package cache
Ponytail	Package plugin	`opencode.jsonc` → `plugins`
OpenWolf	Local plugin file	`plugins/openwolf.js`
OpenSpec	Standalone CLI, not a plugin	`openspec/` per project
Herdr	Standalone CLI, not a plugin	`~/.local/bin/herdr`

`opencode plugin list` only reports managed package plugins. It will not show local files in `plugins/` or TUI plugins. Check the three surfaces directly: the `plugins` array in `opencode.jsonc`, the `plugin` array in `tui.jsonc`, and the contents of `~/.config/opencode/plugins/`.

After installing, start `opencode` once and run `/mcps` if you use MCP servers.

Permissions and policies: see section 10 for the security and permissions chapter. Updates are `opencode upgrade`, `npm i -g @fission-ai/openspec@latest`, `herdr update`, and `opencode plugin update`.

---

3. Where everything lives

Path	What it is
`~/.config/opencode/opencode.jsonc`	Main OpenCode config: providers, models, plugins
`~/.config/opencode/tui.jsonc`	TUI settings
`~/.config/opencode/cli.json`	CLI settings
`~/.config/opencode/oh-my-opencode-slim.json`	Agent presets, models, council seats
`~/.config/opencode/plugins/`	Auto-loaded plugins (`openwolf.js`, `herdr-agent-state.js`)
`~/.config/opencode/.ponytail-active`	Current Ponytail level (`full`)
`~/.config/ponytail/config.json`	Optional Ponytail default mode
`~/.config/openspec/config.json`	Global OpenSpec config
`~/.config/herdr/config.toml`	Herdr config

Path	What it is
`~/.config/herdr/herdr.sock`	Herdr server socket
`.wolf/` (per project)	OpenWolf project memory
`openspec/` (per project)	OpenSpec specs and changes

---

4. OpenCode — the host

Start

```
opencode                 # TUI in the current directory
opencode /path/to/proj   # TUI in another directory
opencode -c              # continue the last session
opencode -s <session-id> # continue/create a specific session
opencode run "message"   # one-shot, non-interactive
opencode service start   # v2 background server (API + web)
opencode api ...          # make a request to the running server
```

Useful subcommands:

```
opencode models          # list available models
opencode mcp add <name> --global --url <url>   # e.g. context7 --url https://mcp.context7.com/mcp
opencode mcp list
opencode mcp logout <name>
opencode auth login      # add a provider credential
opencode plugin add @dietrichgebert/ponytail
opencode plugin list
opencode plugin remove @dietrichgebert/ponytail
opencode session list
opencode reload          # reload config
opencode upgrade
opencode stats
opencode debug --help    # troubleshooting tools
```

Inside the TUI

- Type a request and press Enter. Use `Tab` / the agent switcher to change agent.
- `/` opens the command menu: `/models`, `/sessions`, `/new`, `/init`, `/undo`, `/redo`, `/compact`, `/share`, `/help` (exact set depends on version).
- `Ctrl+P` opens the command palette in most builds.
- Subagents are called by name, e.g. `@explorer find every caller of foo`.

Config

`opencode.jsonc` holds `providers`, `disabled_providers`, `plugin` (v1 key) and `plugins` (v2 key). A typical setup can have both:

```
"plugin":  ["opencode-models-discovery@1.8.0", "oh-my-opencode-slim"],
"plugins": ["@dietrichgebert/ponytail"]
```

Rules of thumb:

- OpenCode 2 uses **`plugins`** (array). OpenCode 1 uses **`plugin`**.
- A path entry must point to a **directory**, never a single `.mjs` file.
- A checkout can be shared with `"/absolute/path/to/checkout/.opencode/plugins"`.
- After editing, run `opencode reload` or restart.

`cli.json` and `tui.jsonc` are separate so you can change UI/CLI behavior without touching providers and models.

---

5. oh-my-opencode-slim — the agent team

Installed as an OpenCode plugin; configuration in `~/.config/opencode/oh-my-opencode-slim.json`. The `free` preset is used here; presets are defined in that file.

Agents

Agent	Use it for
orchestrator (default)	Plans, splits work into lanes, dispatches, reconciles, verifies
@explorer	Read-only search: codebase recon, docs/web research
@oracle	Architecture, risk, debugging strategy, review — escalation only
@designer	All UI/UX: layout, hierarchy, spacing, motion, responsive feel
@fixer	Bounded, well-defined implementation; parallel per-area lanes
@observer	Image/PDF/diagram analysis (pass the full file path)
council	Multi-model consensus, seats `alpha`, `beta`, `gamma`

The orchestrator is lazy about delegating: small, in-context changes it does itself; anything non-trivial goes to a specialist. Parallel lanes are only dispatched when their write scopes do not overlap.

Configuration

The JSON file sets, per preset: model, variant, disabled agents, image routing, multiplexer and the council roster. `preset: "free"` selects the free-model set. Backups (`*.bak-*`) are kept next to it — restore one if an edit breaks things.

To tune agents, models, skills, MCPs or presets, load the `oh-my-opencode-slim` skill in OpenCode and ask; it knows the schema and the safe edit flow.

Calling agents

- Let the orchestrator delegate automatically for non-trivial work.
- Force one directly: `@explorer ...`, `@fixer ...`, `@oracle ...`.
- Council: ask for "a council" on a decision; seats answer in parallel and a synthesis step merges them.

---

6. OpenSpec — decide what to build

OpenSpec turns a request into artifacts: **proposal → specs (delta) → design → tasks**, then implements and archives. It is installed globally (1.14.0) and its skills are available inside OpenCode.

Initialize a project

```
cd /path/to/project
openspec init
```

This creates the `openspec/` directory and the instruction/skill files.

Core loop

```
openspec list                # active changes
openspec list --specs        # current specs
openspec view                # interactive dashboard
openspec show <change>       # one change
openspec status <change>     # artifact completion
openspec validate <change>   # structural validation
openspec archive <change>    # archive + merge deltas into main specs
openspec doctor              # health of the OpenSpec root
openspec context             # print working context
```

Change management:

```
openspec change show <name>
openspec change validate <name>
openspec spec list
openspec spec show <spec-id>
openspec spec validate <spec-id>
```

Skills inside OpenCode

These are the workflow shortcuts (invoke them by name or just ask in plain words):

Skill	Purpose
`openspec-new-change`	Start a change (proposal first)
`openspec-continue-change`	Create the next artifact
`openspec-ff-change`	Fast-forward all artifacts in one pass
`openspec-propose`	Full proposal (design + specs + tasks) at once
`openspec-apply-change`	Implement the tasks
`openspec-verify-change`	Check implementation against artifacts

Skill	Purpose
`openspec-archive-change`	Archive a completed change
`openspec-bulk-archive-change`	Archive several at once
`openspec-sync-specs`	Merge delta specs without archiving
`openspec-update-change`	Revise a plan and keep artifacts coherent
`openspec-explore`	Thinking partner before/while a change
`openspec-onboard`	Guided end-to-end walkthrough

Example

```
You:  "openspec new change: add CSV export to the reports page"
AI:   proposal → specs → design → tasks
You:  "openspec apply"
AI:   implements task by task (Ponytail rules apply)
You:  "openspec verify" then "openspec archive"
```

Use OpenSpec when the work deserves a paper trail. For a one-line fix, skip it — that is the Ponytail rule too.

---

7. Ponytail — decide how little to build

Run this stack at **full** (the active level is stored in `.ponytail-active`). It is one prompt that shapes every coding reply:

1. Does this need to exist at all? (YAGNI)
2. Is it already in this codebase?
3. Does the standard library do it?
4. Is there a native platform feature?
5. Is there an installed dependency?
6. Can it be one line?
7. Only then: the minimum code that works.

Never cut: trust-boundary validation, error handling that prevents data loss, security, accessibility, or anything you explicitly asked for. Non-trivial logic leaves one small test behind. Every reply ends with what was skipped and the risk.

Levels

Level	Behavior
`lite`	Build what was asked; name the lazier option in one line
`full` (default)	The ladder above
`ultra`	Deletion before addition; challenges the requirement before building
`off`	Disabled

Commands

Command	What it does
`/ponytail`	Report level; turn on at default if off
`/ponytail lite|full|ultra|off`	Switch level
`/ponytail-review`	Review current diff: bugs, security, load, tests, speed, cuts
`/ponytail-audit`	Same checks for the whole repo, ranked
`/ponytail-debt`	Collect `shortcut:` comments into a ledger
`/ponytail-gain`	Benchmark scoreboard
`/ponytail-help`	Quick reference

Deactivate with `stop ponytail`, `normal mode`, or `/ponytail off`. Resume with `/ponytail`.

The full discipline rules live in section 11.

Defaults

Resolution order: `PONYTAIL_DEFAULT_MODE` env var → `~/.config/ponytail/config.json` (`{"defaultMode": "lite"}`) → `full`.

`shortcut:` comments

A deliberate simplification gets:

```
// shortcut: supports one level of nesting, upgrade when the tree editor lands
```

`/ponytail-debt` harvests these so “later” does not become “never”.

---

8. OpenWolf — project memory

Plugin: `~/.config/opencode/plugins/openwolf.js` (auto-loaded). Data lives in `.wolf/` in each project.

File	Contents
`.wolf/OPENWOLF.md`	Rules the agent follows
`.wolf/anatomy.md`	File descriptions/sizes — check before reading a file
`.wolf/cerebrum.md`	Do-not-repeat rules — check before writing

File	Contents
`.wolf/buglog.json`	Logged bugs/fixes
`.wolf/memory.md`	General memory
`.wolf/token-ledger.json`	Token accounting

Current releases (2.5+) use `.wolf/anatomy-index.json`; older installs use `anatomy.md`. The workflow is identical — check whichever file exists.

Workflow it enforces:

1. Before reading a file, check the anatomy file.
2. Before writing, check `cerebrum.md`.
3. After fixing a bug, log it (`wolf_log_bug`).
4. When corrected, record it (`wolf_learn`).
5. Before debugging, search known fixes (`wolf_search_bugs`).

If `.wolf/` is missing in a project, the plugin starts it. If you work in many repos, let each one build its own memory — do not share `.wolf/` between projects.

---

9. Herdr — terminal workspace manager

Binary: `~/.local/bin/herdr`. It runs a local server and keeps sessions alive across terminal restarts.

Start

```
herdr                          # launch or attach to the persistent session
herdr --session <name>         # named session
herdr status                   # client/server status
herdr server stop              # stop the server
herdr server reload-config     # apply config.toml changes
herdr update                   # self-update
herdr channel set stable|preview
```

Core objects

`session → workspace → tab → pane → agent`

```
herdr session attach <name>
herdr workspace list|create|get|focus|rename|close
herdr worktree list|create|...        # git worktree helpers
herdr tab ...
herdr pane ...
```

Agent control (the useful part)

```
herdr agent list                      # running agents in panes
herdr agent get <agent>
herdr agent read <agent>              # read terminal output
herdr agent prompt <agent> "..."      # submit a prompt
herdr agent send-keys <agent> ...     # send key presses
herdr agent wait <agent> --state <s>  # block until a state
herdr agent start <agent>             # start a supported agent in a pane
herdr agent attach <agent>            # attach directly
```

Typical setup: one workspace per project, one pane running `opencode`, more panes running tests/servers. Herdr keeps them alive; the OpenCode plugin `herdr-agent-state.js` mirrors agent state into the session.

Remote and multi-machine

```
herdr --remote <ssh-target> [--session <name>]
herdr machine add|list|remove ...
```

Config: `~/.config/herdr/config.toml` (UI, onboarding and keybindings). Agents can ask Herdr for its own skill with `herdr --skill`; human setup guide: https://herdr.dev/agent-guide.md.

---

10. Security and permissions — what the agent may touch

The agent runs as **you**. Every tool call uses your UID, your filesystem, your network, your SSH agent and any cloud token in your environment. Permissions reduce accidents. They do not contain a hostile prompt.

10.1 The trust model

- `read`, `edit` and `shell` reach anything your user can reach.
- `external_directory` **defaults to allow** outside the project — add an `ask` rule if you want a prompt.
- An `ask` can be approved as “always”, which stores a durable allow rule. A `deny` is never overridden by that.
- `policies` run last and can only hard-deny. They are the only layer a saved approval cannot loosen.
- Shell checks use a best-effort scanner. Compound commands, `eval`, and anything dynamic are where it stops being reliable.

10.2 A starter policy

Put shared rules at the top level, narrow rules after them (last match wins):

```
"permissions": [
  { "action": "shell", "resource": "*", "effect": "ask" },
  { "action": "shell", "resource": "git status *", "effect": "allow" },
  { "action": "shell", "resource": "git diff *", "effect": "allow" },
  { "action": "shell", "resource": "git log *", "effect": "allow" },
  { "action": "read", "resource": "*.env", "effect": "deny" },
  { "action": "read", "resource": "*.env.*", "effect": "deny" },
  { "action": "read", "resource": "*.ssh/*", "effect": "deny" },
  { "action": "read", "resource": "*.pem", "effect": "deny" },
  { "action": "read", "resource": "*.config/gh/*", "effect": "deny" },
  { "action": "external_directory", "resource": "*", "effect": "ask" },
  { "action": "shell", "resource": "tofu apply *", "effect": "ask" },
  { "action": "shell", "resource": "tofu destroy *", "effect": "deny" },
  { "action": "shell", "resource": "ansible-playbook *", "effect": "ask" }
],
"policies": [
  { "action": "tool.use", "resource": "shell:sudo *", "effect": "deny" },
  { "action": "tool.use", "resource": "shell:ssh *", "effect": "deny" },
  { "action": "tool.use", "resource": "shell:gh auth token*", "effect": "deny" },
  { "action": "tool.use", "resource": "shell:docker system prune *", "effect": "deny" },
  { "action": "tool.use", "resource": "shell:git push --force*", "effect": "deny" }
]
```

Paths in `read` and `external_directory` rules expand `~`, `$HOME` and `$HOME/`. Shell resources do **not** — write them as plain command text.

10.3 Keys and secrets

A **public** key is not a secret. Paste it into the chat as text when you want it reviewed; no file access and no shell needed.

A **private** key is different. Never paste it, never put it in the project, and assume anything that reaches a model is logged. If it leaks, rotate it — deleting the message is not enough.

If your keys are loaded in `ssh-agent`, any shell command can sign with them. Starting OpenCode without the agent socket removes that capability:

```
env -u SSH_AUTH_SOCK opencode
```

Provider keys belong in `{file:...}` indirection, not inline in the config. The value is a path such as `secrets/provider.key`.

10.4 MCP servers are context and trust

- Every server injects its tool schemas into **every** prompt. Three to five servers is a working ceiling; ten is self-harm.
- Prefer a read-only token with the smallest scope. For GitHub, restrict toolsets to `repos`, `pull_requests`, `actions`, `code_security`.
- A local MCP server runs with your authority and is a supply-chain dependency. Install only what you would run yourself.
- Never give a generic database MCP a production credential.

10.5 What this does not do

Permissions are guardrails, not a sandbox. A container or VM with no key material, no cloud credentials and no network is the only real isolation for untrusted code or untrusted input. Treat repository files and web pages as untrusted input: a prompt injection can ask the agent to run the commands you left allowed.

One more trap: `!` shell blocks inside custom commands execute **before** the prompt is submitted and outside the normal permission flow. Never build a command that pastes untrusted arguments into one.

---

11. When not to use the stack

The failure mode of a good workflow is ceremony. These are the cases where the right move is to drop a layer.

Situation	Skip	Why
One-line fix, obvious cause	Everything except plain ask	A spec costs more than the diff
Spike or unknown problem	OpenSpec	You cannot specify what you have not found yet
Throwaway script	OpenSpec, council	No maintenance, no audience, no paper trail
Context already near the limit	MCP servers, extra subagents	Every tool schema and child session spends context
Production credentials in play	The whole stack	Use a short-lived token in a container, or do it yourself
Latency-critical loop	Parallel lanes, council	Model calls dominate; parallelism multiplies spend
Repo has strong `AGENTS.md`	Ponytail defaults	Project conventions win; drop to `lite`

Three rules of thumb:

1. **If a CLI already does it, use the CLI.** A scanner does not need an MCP server; `tofu validate`, `ansible-lint` and `trivy` are shell commands.
2. **If the answer fits in one sentence, do not open a change.** OpenSpec earns its cost when the work spans days or people.
3. **Every subagent is a fresh context.** A council of three plus a verifier is four full reads of the problem. Use it for decisions that are expensive to get wrong, not for routine edits.

The stack is at its best on multi-file changes with a review gate. On a 20-line fix it should feel like nothing is there.

---

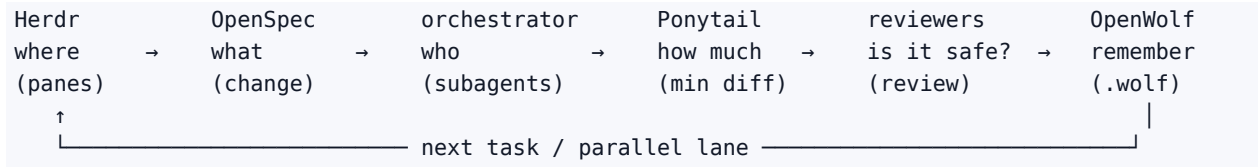
12. Working with all six together — maximum gain

12.1 Who decides what

Each tool owns one decision. Overlap is what makes stacks feel heavy; keep the jobs separate.

Decision	Owner	Not the others
Where the work happens (terminals, panes, isolation)	**Herdr**	OpenCode stays inside one pane
What is being built (scope, acceptance, proof)	**OpenSpec**	the agent does not invent scope
Who does the work (plan, delegate, review)	**oh-my-opencode-slim**	you do not micromanage lanes
How much code is written	**Ponytail**	no gold-plating, no stubs
What was learned (bugs, rules, file map)	**OpenWolf**	no re-debugging the same bug

12.2 The loop



Start in Herdr, decide scope with OpenSpec when it is worth it, let the orchestrator route the work, let Ponytail keep it small, review with the Ponytail checkers, and let OpenWolf keep the lessons.

12.3 Weight the process to the task

Process is a cost. Match it to the size of the change.

Size	Process	Commands
One-line fix / typo	none	just ask; Ponytail full
Small fix, few files	light	ask → `/ponytail-review`
Feature, multi-file	spec	OpenSpec change → apply → verify → archive
Big / risky / parallel	spec + isolation	OpenSpec + Herdr worktrees + parallel `@fixer` lanes

The most common failure is running the big process on small work. The second is skipping the spec on a feature and re-discovering the requirements in review.

12.4 Scenario A — small fix (minutes)

```
You (in OpenCode):  "The export button shows the wrong date format. Fix it."
AI:                 finds the formatter, reuses the existing helper, one-line diff
You:                /ponytail-review
AI:                 flags nothing, or names the real risk
```

- No OpenSpec, no council, no Herdr worktree.
- OpenWolf quietly records the correction if it was a repeat mistake.
- If the fix needed a shortcut, it leaves a `shortcut:` comment; `/ponytail-debt` will collect it later.

12.5 Scenario B — feature with a spec

```
You:  "openspec new change: add CSV export to reports"
AI:   proposal → delta specs → design → tasks        (OpenSpec artifacts)
You:  "openspec apply"
AI:   implements task by task at Ponytail full        (smallest complete change)
You:  "/ponytail-review"                              (diff-level quality gate)
You:  "openspec verify"
AI:   checks the implementation against the artifacts
You:  "openspec archive"
AI:   merges the delta into the main specs
```

Where the gain comes from: the spec stops scope creep *before* code exists, and Ponytail stops the spec from turning into an architecture essay. Verify catches “implemented but not what was asked” — the failure mode plain code review misses.

12.6 Scenario C — parallel lanes (big change)

Use Herdr when one agent is the bottleneck.

```
herdr                                # start/attach the session
herdr workspace create myapp         # one workspace per project
herdr worktree list                  # see existing isolated checkouts
```

- Create one worktree per independent lane (API, UI, tests).
- Run one OpenCode per worktree, each in its own pane.
- Give each lane a non-overlapping write scope. Two agents editing the same file is not parallelism, it is a merge conflict with extra steps.
- Drive them from one place:

```
herdr agent list
herdr agent prompt <agent> "implement task 3 from the change"
herdr agent wait <agent> ...          # see: herdr agent wait --help
```

- Reconcile the lanes yourself (or with `@oracle`) before the final review.
- Merge only after `/ponytail-review` passes in each lane.

12.7 Scenario D — the bug that keeps coming back

```
You:  "We fixed this before — check known bugs"
AI:   OpenWolf wolf_search_bugs / .wolf/buglog.json
You:  "@explorer trace every caller of X"
AI:   read-only recon, no edits
You:  (if still stuck) "@oracle, two attempts failed — strategy?"
AI:   architecture/debugging escalation, not another patch
You:  fix + one small test
AI:   wolf_log_bug records the fix; cerebrum.md records the rule
```

This is the highest-leverage loop in the stack: OpenWolf turns one expensive debugging session into a permanent asset.

12.8 Power combos — what each pairing buys you

Combination	Gain
Herdr + OpenCode	Sessions survive restarts; several agents work at once in separate panes
Herdr worktrees + @fixer	True parallel implementation with no shared working tree
oh-my-opencode-slim + Ponytail	A team that delegates, but every lane writes the minimum
OpenSpec + Ponytail	Scope fixed up front, code kept small; neither turns into ceremony
OpenSpec + @oracle	Design decisions get challenged before they become specs
OpenWolf + @explorer	Fewer files read, fewer tokens spent mapping the repo
Ponytail-review + @oracle	Cheap review first; escalate only the findings that matter
Council + @oracle	Multi-model consensus for irreversible decisions (schema, auth, migrations)
Ponytail-debt + OpenWolf	Deferred shortcuts and known bugs become one visible backlog

Goal	Say

12.9 Phrasebook — what to type

Goal	Say
Understand before editing	“Look around first, then fix it” (or `@explorer`)
Smallest solution	“/ponytail full” then describe the task
Challenge the requirement	“/ponytail ultra”
Fix without scope creep	“Fix only X; do not refactor around it”
Start tracked work	“openspec new change: ”
Review the diff	“/ponytail-review”
Repo health check	“/ponytail-audit”
Deferred work	“/ponytail-debt”
Second opinion	“@oracle, is this the right design?”
Consensus	“Run the council on this decision”
Parallel work	“Split into lanes with non-overlapping files”
Remember a fix	“Log this bug and the rule to avoid it”

12.10 Guardrails — what not to do

- **Do not run OpenSpec on a one-line fix.** The artifacts cost more than the change.
- **Do not set ultra as the default.** It questions the request every time; use it when a ticket feels bloated, not for routine work.
- **Do not run two writers on the same files.** Use worktrees or serialize.
- **Do not let `.wolf/` go stale.** Prune `memory.md` when it contradicts reality; a wrong memory is worse than none.
- **Do not let the orchestrator delegate a change you already fully described.** Small, in-context work should stay in the main session.
- **Do not skip the test on logic with branches, money or security.** Ponytail never cuts that.
- **Do not archive a change you have not verified.** `openspec verify` first.

12.11 Routine

Every session

- Open Herdr, attach the project workspace, start OpenCode.
- Check `/ponytail` reports `full`.
- Let OpenWolf answer “have we seen this before?”.

Weekly

- `/ponytail-debt` — harvest and triage `shortcut:` comments.
- `openspec list` — archive or close stale changes.
- `/ponytail-review` on the week’s diff before it grows.

Monthly / before a release

- `/ponytail-audit` — whole-repo ranking, fix the top item first.
- `openspec doctor` + `openspec validate` on active specs.
- Prune `.wolf/memory.md` and `cerebrum.md`; keep rules that still hold.
- `herdr status`; stop stale sessions and workspaces.

12.12 Which tool when — the one-line rule

- New feature or behavior change → **OpenSpec**.
- Small fix or refactor → **skip OpenSpec**, just ask (Ponytail full).
- “Is this slow / safe / over-engineered?” → `/ponytail-review`, escalate to **@oracle** only if it matters.
- Several terminals, long-running agents, isolated branches → **Herdr**.
- “We fixed this before” → **OpenWolf**.
- Team of agents doing the work → **oh-my-opencode-slim** (default orchestrator).

---

13. Cheat sheet

One block per tool. Copy-paste these, do not memorize them.

```
# OpenCode
opencode                      # TUI
opencode run "..."            # one-shot
opencode models               # models
opencode mcp add <name> --global --url <url>   # e.g. context7 --url https://mcp.context7.com/mcp
opencode mcp list
opencode auth login           # add a provider credential
opencode plugin add @dietrichgebert/ponytail
opencode plugin list
opencode plugin remove @dietrichgebert/ponytail
opencode session list
opencode reload               # reload config
opencode upgrade              # update
opencode update               # update to a specific version
opencode uninstall            # uninstall, keep data/config
opencode acp                   # ACP server
opencode debug agents         # list agents
opencode stats                # usage stats
opencode mini                 # minimal interface

# oh-my-opencode-slim (inside OpenCode)
@orchestrator <task>          # default: plan, split lanes, dispatch, verify
@explorer <task>              # read-only recon
@oracle <task>                # escalation
@designer <task>              # UI/UX
@fixer <task>                 # bounded implementation
@observer <task>              # reviews / images / diagrams
@council <task>               # multi-model consensus

# OpenSpec
openspec init                 # set up project
openspec new change <name>  # openspec/changes/<name>
openspec list                 # changes
openspec list --specs         # current specs
openspec view                 # dashboard
/openspec-explore             # explore mode: think through ideas before implementing
openspec validate <change>
openspec status <change>      # artifact completion
openspec archive <change>
openspec doctor

# Ponytail (inside OpenCode)
/ponytail lite|full|ultra|off
/ponytail-review
/ponytail-audit
/ponytail-debt

# Herdr
herdr                         # launch/attach
herdr status
herdr workspace list
herdr workspace create <name>
herdr agent list
herdr agent start <agent>
herdr agent prompt <agent> "..."
herdr --skill                 # give an agent Herdr's own instructions
```

14. Troubleshooting

Symptom	Check
Plugin not loaded	Path must be a directory; OpenCode 2 uses `plugins`, OpenCode 1 uses `plugin`; then `opencode reload`
Ponytail seems inactive	`cat ~/.config/opencode/.ponytail-active`; `/ponytail` should report `full`
Ponytail fights `AGENTS.md`	The repo's own `AGENTS.md` wins on project conventions; set level `lite` if it is too aggressive
Agent team missing	`~/.config/opencode/oh-my-opencode-slim.json` valid JSON; preset exists; restart OpenCode
OpenSpec odd state	`openspec doctor`, then `openspec validate` on the change
Herdr config ignored	`herdr server reload-config`; full restart if a keybinding changed
Duplicate/stale Herdr state	`herdr status`, then `herdr server stop` and relaunch
Hook errors mentioning `node`	Install Node or put it on the non-interactive shell PATH

---

15. Appendix A — templates to copy

A. Minimal `opencode.jsonc`

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "provider/model",
  "small_model": "provider/model-small",
  "lsp": true,
  "plugins": ["@dietrichgebert/ponytail"],
  "permissions": [
    { "action": "shell", "resource": "*", "effect": "ask" },
    { "action": "read", "resource": "*.env", "effect": "deny" },
    { "action": "external_directory", "resource": "*", "effect": "ask" }
  ]
}
```

`plugins` is the V2 array. Local plugin files live in `~/.config/opencode/plugins/` and load automatically. TUI plugins use the separate `plugin` array in `tui.jsonc`.

B. Project `AGENTS.md`

```
# AGENTS.md

## Stack
OpenCode + oh-my-opencode-slim, OpenSpec for changes, Ponytail for minimal
diffs, OpenWolf for project memory.

## Commands
- test: `npm test`
- lint: `npm run lint`
- format: `npm run format`

## Rules
- Check `.wolf/anatomy.md` before opening files and `.wolf/cerebrum.md` before writing.
- No new dependency without asking.
- Never read or print `.env*`, `*.pem`, or anything under `~/.ssh`.
- Do not run `tofu apply`, `ansible-playbook`, `docker system prune`, or force-push.
- Match the surrounding style. No drive-by refactors.
```

The repo's `AGENTS.md` wins over Ponytail defaults on project conventions. Keep it short; every line costs context in every session.

C. `/security-review` command

Save as `commands/security-review.md` under `~/.config/opencode/` (global) or `.opencode/` (project):

```
---
description: Review the current diff for security defects
---

Review the staged and unstaged changes for security defects.

1. Run `git diff` and `git diff --staged`.
2. Check for secrets in code, injection (shell, SQL, template), unsafe
   deserialization, path traversal, missing authorization, permissive CORS,
   unpinned dependencies, and over-broad CI permissions.
3. For each finding: file and line, severity, why it matters, and the smallest
   fix.
4. Do not edit files. Do not run the application.
```

No `!` shell block: those run outside the permission flow. Let the agent call the tools so every command is checked.

D. OpenSpec change skeleton

```
openspec new change add-csv-export
```

That creates `openspec/changes/add-csv-export/`:

File	Fill it with
`proposal.md`	Problem, why now, scope, non-goals
`specs/<capability>/spec.md`	Delta requirements: ADDED / MODIFIED / REMOVED
`design.md`	Decisions, alternatives, risks (only if the change is not obvious)

File	Fill it with
`tasks.md`	Ordered, checkable implementation steps

Then validate, implement, verify, and archive: `openspec validate <change>`, `openspec verify`, `openspec archive <change>`.

---

16. Appendix B — glossary

Term	Meaning
Agent	A configured assistant with its own prompt, model and permissions
Subagent	A child session launched by another agent for one bounded task
Tool call	One action the model requests: read, edit, shell, fetch, MCP tool
MCP	Model Context Protocol — how external servers expose tools to the agent
Context window	The tokens a model can see at once; every tool schema and file spends it
Compaction	Automatic summarization when a session approaches the context limit
Permission	Ordered allow/ask/deny rule for an action on a resource
Policy	A hard-deny layer that saved approvals cannot override
Worktree	A second checkout of the same repo on another branch, used for parallel lanes
Change	OpenSpec's unit of work: proposal, delta specs, design, tasks
Delta spec	Requirements added, modified or removed by one change
Archive	Merging a finished change's deltas into the main specs
Preset	A named model/agent configuration in oh-my-opencode-slim
Pane	One terminal inside a Herdr tab, usually running one agent

Tool	Tested	License	Source

17. Appendix C — versions, licenses and sources

Tested on this setup in October 2026. Licenses were checked at the same time; confirm them before redistributing anything.

Tool	Tested	License	Source
OpenCode	2.2.1	MIT	repo · docs
oh-my-opencode-slim	3.0.6	MIT	repo · npm
OpenSpec	1.14.0	MIT	repo · docs
Ponytail	5.3.3	MIT	repo · install
OpenWolf	plugin, Oct 2026	AGPL-3.0	repo · docs
Herdr	0.9.3	Apache-2.0	repo · docs

Two naming traps worth knowing:

- **oh-my-opencode-slim is not oh-my-opencode.** The original `oh-my-opencode` is a different project by a different author under the SUL-1.0 license. Slim is the MIT fork used here.
- **OpenWolf is not the only project with that name.** The one used here is `cytostack/openwolf`, the OpenCode plugin that creates `.wolf/`.

The guide text is original. The tools are third-party; link their repositories rather than implying authorship.